

PROJETO WISE WARRIOR

Documento de Arquitetura de Software

WW – Wise Warrior	
Equipe	Instituição
Grupo 2	UCB
Engenharia de Software	16/04/2026

Objetivo deste Documento
Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento do Wise Warrior e os critérios considerados durante a tomada dessas decisões. Suas informações abrangem a estrutura lógica, física e tecnológica do sistema, contemplando componentes de <i>software</i> , comunicação e persistência.

Histórico de Revisão				
Data	Demanda	Autor	Descrição	Versão
16/04/2026	WW-ARQ-001	Grupo 2	Versão inicial do documento de arquitetura do Wise Warrior.	1.0

Sumário

1.	DESCRIÇÃO GERAL DA ARQUITETURA	2
2.	ESTILO ARQUITETURAL ADOTADO	3
3.	PRINCIPAIS COMPONENTES E RESPONSABILIDADES	3
3.1	Camadas do Backend	3
3.2	Módulos do Backend	3
3.3	Frontend (SPA React)	4
4.	DIAGRAMA DE ARQUITETURA	4
5.	TECNOLOGIAS PREVISTAS	5
6.	JUSTIFICATIVA DAS DECISÕES ARQUITETURAIS	6

1. DESCRIÇÃO GERAL DA ARQUITETURA

O Wise Warrior é uma plataforma web gamificada de foco e produtividade colaborativa, na qual usuários realizam sessões de estudo (Pomodoro), ganham XP, sobem de nível, formam guildas e participam de raids semanais.

Do ponto de vista arquitetural, o sistema é composto por quatro partes principais:

- **Cliente Web (SPA)** — aplicação React/TypeScript executada no navegador do usuário, responsável pela interface gamificada (timer, dashboards de XP, chat, ranking de guildas).
- **Servidor de Aplicação (API + Tempo Real)** — backend monolítico modular em NestJS que expõe uma API REST e um servidor WebSocket (Socket.IO).
- **Banco de Dados Relacional** — MySQL 8, responsável pela persistência de usuários, sessões, progressão, guildas, raids e mensagens.
- **Cache / Pub-Sub** — Redis, usado como adapter do Socket.IO quando há múltiplas réplicas do backend.

O sistema é implantado em nuvem pública (AWS) e acessado via HTTPS/WSS.

2. ESTILO ARQUITETURAL ADOTADO

O Wise Warrior adota uma combinação de Cliente-Servidor com Monólito Modular em Camadas:

- **Cliente-Servidor** — descreve a distribuição física: um cliente web consome um servidor centralizado via HTTP/WebSocket.
- **Monólito Modular** — o backend é publicado como um único artefato (container Docker), mas internamente dividido em módulos coesos e baixo acoplamento entre si.
- **Camadas** — a organização interna do código segue quatro camadas horizontais: Apresentação → Aplicação → Domínio → Infraestrutura.

Por que não microserviços? Trata-se de um produto universitário de ~1 ano, desenvolvido por uma equipe de 4 pessoas, cujo domínio é fortemente acoplado (sessão → XP → progressão → raid → guilda). Microserviços trariam complexidade operacional desnecessária. A modularização interna mantém a porta aberta para extração futura sem reescrita.

3. PRINCIPAIS COMPONENTES E RESPONSABILIDADES

3.1 Camadas do Backend

Camada	Responsabilidade
Apresentação / API	Receber requisições HTTP/WS, validar DTOs, serializar respostas, controlar sessão do WebSocket. Ex.: SessionController, RaidGateway, AuthGuard.
Aplicação	Orquestrar casos de uso, controlar transações, coordenar chamadas entre módulos. Ex.: StartSessionService, ContributeToRaidService.
Domínio	Regras de negócio puras, invariantes, cálculo de XP/nível, condições de desbloqueio. Ex.: Session, Character, ProgressionPolicy.
Infraestrutura	Persistência, envio de e-mail, adapters de terceiros. Ex.: SessionRepository, TypeOrmUnitOfWork.

Tabela 1 — Camadas do backend e responsabilidades

Regra de dependência: Controllers nunca acessam repositórios diretamente — sempre passam por um service da camada de aplicação. A camada de domínio não conhece infraestrutura.

3.2 Módulos do Backend

Módulo	Responsabilidade principal
auth	Cadastro, login, emissão e validação de JWT, recuperação de senha, guards de rota.
users	Perfil do usuário, inventário de itens cosméticos, customização do personagem.
sessions	Temporizador Pomodoro, registro de sessões concluídas, histórico por usuário.

Módulo	Responsabilidade principal
progression	Cálculo de XP, regras de nível, desbloqueio de itens/títulos, motor de recompensas.
guilds	Criação e gestão de guildas, membresia, ranking interno de contribuição.
raids	Raids semanais, metas coletivas, contribuições individuais, conclusão.
chat	Chat em tempo real por guilda (persistência das últimas N mensagens).
notifications	Emissão de eventos em tempo real (raid concluída, convite de guilda, novo nível).
shared	Tipos comuns, utilitários, filtros de exceção, interceptadores transversais.

Tabela 2 — Módulos do backend e responsabilidades principais

Regra de acoplamento entre módulos: a comunicação passa sempre pelo Service público do módulo de destino (ex.: sessions chama ProgressionService.awardXp(), nunca XpRepository). Isso viabiliza a extração futura para microserviços sem reescrita.

3.3 Frontend (SPA React)

O frontend é uma SPA em React + TypeScript organizada por features que espelham os módulos do backend (auth, sessions, profile, progression, guilds, raids, chat). A comunicação usa Axios para REST e socket.io-client para tempo real.

4. DIAGRAMA DE ARQUITETURA

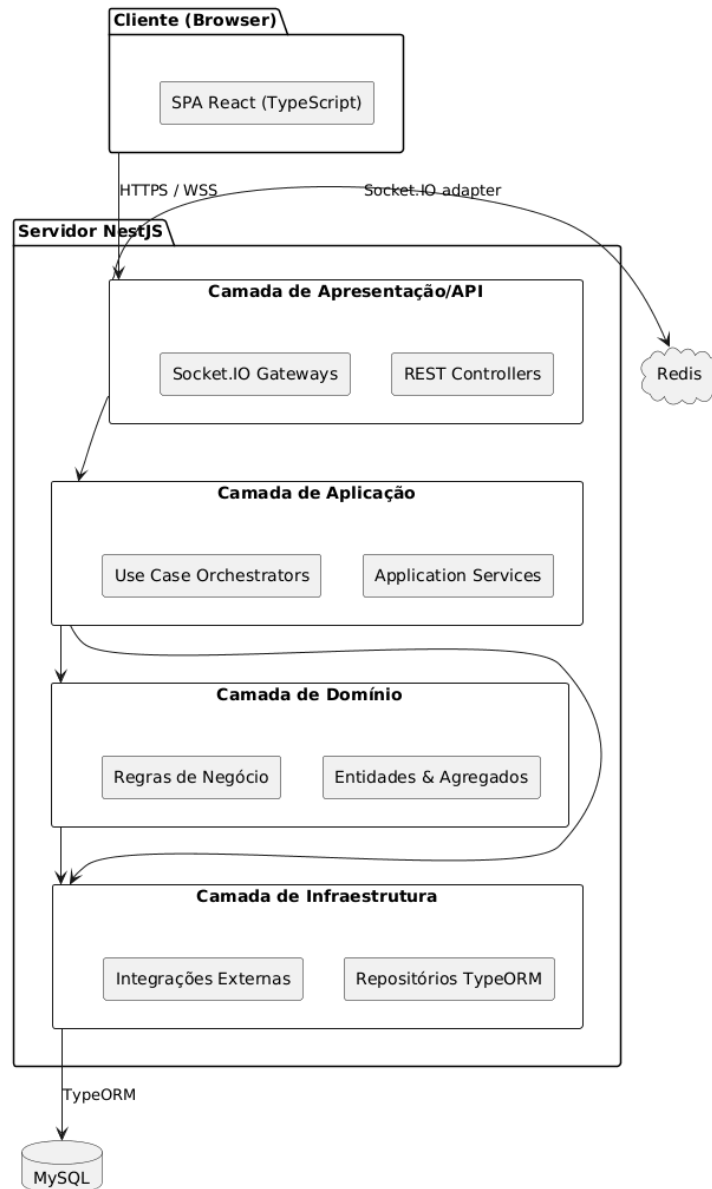


Figura 1 — Diagrama de Camadas da Aplicação

Descrição: as setas representam a direção permitida de dependência (de cima para baixo). Controllers delegam para services; services orquestram domínio e infraestrutura; domínio não conhece infraestrutura. Redis é usado pela camada de apresentação como adapter do Socket.IO para propagar eventos entre réplicas do backend.

5. TECNOLOGIAS PREVISTAS

Camada / Função	Tecnologia
Linguagem (backend e frontend)	TypeScript
Backend — runtime	Node.js (LTS)
Backend — framework	NestJS (REST + Dependency Injection + estrutura modular)
Backend — tempo real	Socket.IO (com Redis Adapter para multi-réplica)
Backend — ORM	TypeORM (migrations versionadas)
Frontend — framework	React 18+

Camada / Função	Tecnologia
Frontend — roteamento	React Router v6
Frontend — HTTP	Axios
Frontend — WebSocket	socket.io-client
Frontend — i18n	react-i18next (pt-BR inicial)
Frontend — build / dev server	Vite
Banco de dados	MySQL 8 (AWS RDS, Multi-AZ)
Cache / Pub-Sub	Redis (AWS ElastiCache)
Infraestrutura	AWS — ECS Fargate, ELB, CloudFront, CloudWatch
Autenticação	JWT (access + refresh), senhas com Argon2id
Transporte	HTTPS / WSS (TLS terminado no ELB)

Tabela 3 — Tecnologias previstas para o projeto

6. JUSTIFICATIVA DAS DECISÕES ARQUITETURAIS

- **Monólito modular em vez de microserviços.** Equipe pequena (4 pessoas), prazo curto (1 ano) e domínio fortemente acoplado. Microserviços adicionariam complexidade de rede, observabilidade e orquestração sem benefício proporcional. A modularização interna preserva a possibilidade de extração futura.
- **TypeScript end-to-end.** A mesma linguagem no backend (NestJS) e no frontend (React) permite compartilhar DTOs/tipos, reduzindo divergências de contrato e erros de integração.
- **NestJS como framework backend.** Estrutura opinativa baseada em módulos, providers e guards se alinha naturalmente à separação em camadas e módulos exigida pela arquitetura. Dependency Injection nativa facilita testes e substituição de implementações.
- **React + Vite no frontend.** React tem o maior ecossistema e material de aprendizado — vantagem importante para equipe universitária. A UI gamificada (timers, barras de XP, inventário, chat) se beneficia fortemente de componentização. Vite oferece build rápido e HMR para produtividade de desenvolvimento.
- **MySQL relacional em vez de NoSQL.** O domínio é inerentemente relacional: usuários possuem sessões, sessões geram XP, XP compõe progressão, progressão habilita itens e títulos, usuários pertencem a guildas, guildas executam raids. Integridade referencial e transações ACID são requisitos naturais do negócio.
- **Socket.IO + Redis Adapter para tempo real.** Chat de guilda, ranking ao vivo e notificações de raid exigem comunicação bidirecional de baixa latência. Socket.IO abstrai WebSocket com fallback. O Redis Adapter é obrigatório a partir de duas réplicas do backend para garantir broadcast consistente.
- **AWS ECS Fargate.** Executa containers Docker sem gestão de instâncias EC2. Permite autoscaling horizontal por CPU/memória e se integra naturalmente com ELB (HTTPS/WSS), CloudWatch (logs/métricas) e RDS (banco gerenciado com failover Multi-AZ).
- **Arquitetura em camadas com módulos coesos.** Controllers nunca acessam repositórios diretamente; cross-module calls passam pelo Service público do módulo destino. Essa disciplina arquitetural protege o domínio de detalhes de infraestrutura e mantém a opção de extrair módulos para microserviços no futuro, sem reescrever regras de negócio.